Execute/Resume Script

Available in the File menu, the **Execute Script** command instructs RegiStar to run a list of commands, comprising a *script*, one after the other. Select the script in the **Open Script File** dialog.

By invoking this command, you can have RegiStar automatically run a specified sequence of operations, without any intervention, and inspect the results after the script has completed. You can also instruct the script to pause at specified points so that you can study intermediate results.

A script enters suspended mode when a Suspend command in the script is executed. If a script is suspended, you can resume execution of the script using the **Resume Script** command. The File menu displays the Resume Script command if a script is suspended and the Execute Script command if no script is running.

When a script is executing, its file name and status — *Running* or *Suspended* — are reported in the upper status bar. You cannot start another operation while a script is executing, even if it is suspended. However, if a script is suspended then some dialogs become available.

The sequence of commands must be saved in an ordinary text file which you can specify in the *Open Script File dialog*. See Script files, Scripting commands, and Scripting directives for instructions on writing this script file.

When a script is run, it passes through two phases. First, RegiStar validates the script without actually executing it, checking for errors and that the directories and files referred to in the script exist. If the script passes this validation, it proceeds to the execution phase. You can specify in the script that it is to be executed if there are no validation-time anomalies other than missing files. If you choose this option, you must ensure that the necessary files exist by the time they are encountered by the script. Execution of a script stops when a missing file is encountered.

Confirmation dialogs that appear when a command is run interactively — namely when a large image is about to be created or when a large number of images are about to be opened in bulk — are skipped when that command is run in a script. An out-of-memory crash or long uninterruptible operation may result if a script includes a runaway command.

A log file (.log) in the same directory as the script file and with the same base name is created whenever a script is executed. It records any anomalies encountered during the validation or execution phases. If a script fails validation or does not yield the expected results after it is executed, you should inspect this file. You can view the log file by using any text editor after the script's execution is complete. Some text editors, such as *Notepad* or *Word,* allow you to view this file while the script is running. Other text editors such as *WordPad* may refuse to open the log file while the script is running and that file is being used by RegiStar.

You should ensure that scripting commands that require images to be in the same group satisfy this grouping requirement, as consistency of grouping is not checked during the validation phase. If during the execution phase a command is encountered that fails required grouping, execution of the script stops. If a command cannot be completed for some other reason, for example a requested registration cannot be performed or an image about to be created is too large to fit into available physical or virtual memory, a warning to that effect is recorded in the log file but script execution proceeds nevertheless. However, the script may subsequently terminate with an error if the image that was to have been produced by this incomplete operation is used later in the script.

If an error is encountered while a script is executing, a cautionary message box appears. This message disappears after 20 seconds or when the *OK* button is pressed.

You can stop a script by using the Stop command. This stops the current operation being executed. If you stop a script, you cannot re-enter it midstream. Instead, you must rerun the script in its entirety. You can pause an operation being run by a script, by using the Pause command.

You can fully inspect all open images while a script is in progress but you cannot close any image opened by or produced by the script. You must stop the script first. If you attempt to close an image involved in an active script, a message appears and the request is ignored.

Scripts can also be run using a command line or from within a batch file. See Command-line scripting for details.

Shortcut

Key: CTRL + E

Script files

The script file specified in an **Execute Script** command must contain a valid sequence of commands, corresponding to the various operations that RegiStar can perform, and satisfy certain rules. If the script file does not meet these requirements, which are outlined below, RegiStar refuses to execute the script, or may run the script with unexpected results. See the Execute Script command for more details about how RegiStar validates a script and handles any anomalies in the script.

Example: The following script:

1. opens images `image1.tif` and `image2.tif` from the directory `C:\test images`;
2. registers `image1.tif` to `image2.tif` (locating stars in these images in order to perform the registration if stars have not already been located);
3. combines the resulting image with `image2.tif`;
4. saves the result as `image3.tif`, and finally close all images.

```
SetDirectory( "C:\images" )
A = Open( "image1.tif" )
B = Open( "image2.tif" )
```

```
C = Register( A, B )
D = Combine( B, C )
Save( D, "image3.tif" )
CloseAll( )
```

Images created by a scripting command must be given an identifier (for example `A, B, C, D`) if they are to be subsequently referred to individually. This identifier, or ID, is a string consisting of a combination of upper or lower case letters, numbers, or the underscore (_) character. It may or may not have a relationship to the actual name of the image. An image ID may not start with a number or underscore character. The ID of an image is just used by the script, and allows the image to be referenced in subsequent commands in the script. Most scripting commands only accept image IDs as inputs instead of actual image names. IDs are case sensitive, so that ID1 and id1 are different. An ID cannot be used to denote more than one open image, but can be reused if the image it denotes is closed.

Several scripting commands create sets of images that cannot subsequently be referred to individually in the script, although image sets can be referenced by several scripting commands.

In addition to image IDs, the only other variables that can appear in a script file are string labels created by the Label scripting command. This command assigns a label to a text string, such as an image's name or a directory path, and then you can use this label instead of the literal string in any command that takes a string argument. The same restrictions apply to these label variables that apply to image IDs, except that label variables can be redefined at any time.

Every command must contain an opening left parenthesis, (, and a closing right parenthesis,). Some commands accept optional parameters, enclosed by square brackets `[ ]` just after the name of the command and before the opening left parenthesis. For example, `D = Combine[1,1]( B, C )` would result in images `B` and `C` being combined using the maximum value of corresponding pixels and with dimensions given by the intersection of `B` and `C`, instead of the default average/union method. See Scripting commands for a complete list of scripting commands and their options.

Commands can be split across lines. However names of commands, IDs, and strings enclosed in quotation marks cannot be split. Blank spaces and tabs between fields of a script command are ignored.

In addition to commands, a script can also contain *comments* and *directives*. The special character `#` forces the rest of the line to be ignored. You can insert a remark to the right of a `#`, to document your script. You can also temporarily skip certain commands in your script by inserting a `#` to the left of the command.

A directive is an instruction to RegiStar that controls how the script is executed. Directives must begin with an exclamation mark (!), which must appear at the beginning of the line. See Scripting directives for more details.

In addition to valid commands, comments, or directives, blank lines are also allowed in a script file. Anything else leads to an inexecutable script.

Write your script file with any text editor and save it as text only (.txt). Then specify it when running the **Execute Script** command.

See Sample scripts for a selection of pre-written scripts that can be used as templates for your own particular applications.

Scripting commands

The commands that can appear in a script file that is specified in an Execute Script command are listed below, in alphabetical order, and with the required syntax. Any name that includes the letters *ID* refers to an image's ID. Replace these placeholders with actual image IDs of your choosing.

Optional parameters `a,b,c,...` with values `0,1,2,...` are enclosed in square brackets. **The value `0` always corresponds to the default.** The brackets and parameters may be omitted, in which case defaults (as given in each command's detailed description) are used. If fewer parameters are given than are allowed, they are taken to be the earliest ones in the list and the others assume defaults. To specify a nondefault value, say `2`, for only, say, the parameter `c`, use the parameter sequence `[0,0,2]`.

Note: The default parameter value in earlier versions of RegiStar was not always `0`. As a result, the syntax for the Combine, CropPad, Register, SaveAll, and SaveAs scripting commands is slightly different now. Read the linked pages to ensure that any previous scripts you wrote will still work as intended.

Each command's name must be written exactly as indicated, including case (see below). The forward slash character `/` or the backslash character `\` may be used when specifying paths.

If during the script's execution a command is encountered for which an input image for that command is missing, the script's execution is usually stopped and an error message is written to the log file. The exceptions are the Close, Release, and Reload commands, which are applied to any images that do exist with missing images ignored. Some commands are also not performed or completed in certain other circumstances, and generally this means the script's execution is terminated. See the individual descriptions of the commands for more details.

Click on a command's name in the left column for more details about the usage of the command. See Sample scripts for examples of the usage of most of these commands.

Command and Syntax	Short Description
`ResID =` **`Calibrate`**`[a,b]( SrcID, RefID )`	Calibrate `SrcID` to `RefID` using options `a,b`.
`Calibrate[a,b]( RefID )`	Calibrate all images in the group of `RefID` to `RefID` using options `a,b`.
`ResID =` **`CaptureDisplay`**`( SrcID )`	Capture Display of `SrcID`.
`Close``( Src1ID, Src2ID,... )`	Close `Src1ID, Src2ID,...`

CloseAll( )	Close all images.
CloseAll(SrcID)	Close all images in the group of SrcID.
CloseAll("<Group name>")	Close all images in the named group.
ResID = **Combine**[a,b](Src1ID, Src2ID,...)	Combine Src1ID, Src2ID,... using options a,b.
ResID = Combine[a,b,c](Src1ID, Wt1, Src2ID, Wt2,...)	Combine Src1ID, Src2ID,... using options a,b,c and weights Wt2, Wt2,...
ResID = Combine[a,b](SrcID)	Combine all images in the group of SrcID using options a,b.
ResID = Combine[a,b,c](SrcID, Wt)	Combine all images in the group of SrcID using weight Wt and options a,b,c.
ResID = Combine[a,b]("<Group name>")	Combine all images in the named group using options a,b.
ResID = Combine[a,b,c]("<Group name>", Wt)	Combine all images in the named group using weight Wt and options a,b,c.
ResID = **CropPad**[a,b](SrcID, RefID)	Crop/Pad SrcID to RefID using options a,b.
ResID = CropPad(SrcID)	Crop/Pad SrcID using the *Trim* option.
CropPad[a,b](RefID)	Crop/Pad all images in the group of RefID using options a,b.
CropPad[a,b]("<Group name>")	Crop/Pad all images in the named group using options a,b.
Describe(SrcID)	Write a description of SrcID to the log file.
String_name = **Label**("<string>")	Assign the text string <string> to String_name.
LocateStars[a](SrcID)	Locate stars of SrcID using option a.
ResID = **Merge**(RedID, GreenID, BlueID)	Merge the specified images.
ResID = **Open**[a,b,c,d]("<Image name>")	Open the specified image using options a,b,c,d.
ResID = Open[a,b,c,d]("<Image name>", w, x, y, z)	Open the specified image using options a,b,c,d and raw specifications w, x, y, z.
Open[a,b,c,d]("<Source Dir>")	Open images in the specified directory using options a,b,c,d.
Open[a,b,c,d]("<Source Dir>", w, x, y, z)	Open images in the specified directory using options a,b,c,d and raw specifications w, x, y, z.
OpenGroup(SrcID)	Open all images in the group of SrcID.
OpenGroup("<Group name>")	Open all images in the specified group.
ResID = **Preprocess**[a](SrcID, RefID)	Preprocess SrcID using the dark frame or flat field RefID using option a.
Preprocess[a](RefID)	Preprocess all open compatible images using the dark frame or flat field RefID, with option a.
Preprocess[a]("<Source Dir>", RefID)	Preprocess all compatible images in <Source Dir> using the dark frame or flat field RefID, with option a.
Preprocess[a,b]("<Source Dir>", RefID, "<Dest Dir>")	Preprocess all compatible images in <Source Dir> using the dark frame or flat field RefID, saving the results in <Dest Dir>, with options a,b.
Quit( )	Stop execution of the script.
ResID = **Register**[a,b,c,d,e,f](SrcID, RefID)	Register SrcID to the group of RefID using options a,b,c,d,e,f.
ResID = Register[a,b,c,d,e,f](SrcID, "<Group name>")	Register SrcID to the specified group using options a,b,c,d,e,f.
Register[a,b,c,d,e,f,g,h,i]("<Source Dir>", RefID)	Register all images in <Source Dir> to the group of RefID, using options a,b,c,d,e,f,g,h,i.
Register[a,b,c,d,e,f,g,h,i]("<Source Dir>", RefID, "<Dest Dir>")	Register all images in <Source Dir> to the group of RefID, saving the results in <Dest Dir>, using options a,b,c,d,e,f,g,h,i.
Register[a,b,c,d,e,f,g,h,i]("<Source Dir>", "<Group name>")	Register all images in <Source Dir> to the specified group, using options a,b,c,d,e,f,g,h,i.
Register[a,b,c,d,e,f,g,h,i]("<Source Dir>", "<Group name>", "<Dest Dir>")	Register all images in <Source Dir> to the specified group, saving the results in <Dest Dir>, using options a,b,c,d,e,f,g,h,i.

Release(Src1ID, Src2ID,...)	Release the image data of Src1ID, Src2ID,...
Reload(Src1ID, Src2ID,...)	Reload the image data of Src1ID, Src2ID,...
ResID = **RotateFlip**[a,b](SrcID)	Rotate/flip SrcID using options a,b.
Save(Src1ID, Src2ID,...)	Save Src1ID, Src2ID,...
SaveAll[a,b,c,d,e]("<Dest Dir>")	Save all open images to the specified directory using options options a,b,c,d,e.
SaveAll[a,b,c,d,e]("<Dest Dir>", SrcID)	Save all open images in the group of SrcID to the specified directory using options a,b,c,d,e.
SaveAll[a,b,c,d,e]("<Dest Dir>", "<Group name>")	Save all open images in the specified group to the specified directory using options a,b,c,d,e.
SaveAs[a,b,c](SrcID, "<Image name>")	Save SrcID with the specified name using options a,b,c.
SaveAs[a,b,c](SrcID, "<Dest Dir>")	Save SrcID to the named directory with its default name and image format, using options a,b,c.
SetDirectory("<Directory name>")	Set the current directory.
SetImageGroup(SrcID, "<Group name>")	Set the group of SrcID.
SetImageGroup(SrcID, RefID)	Set the group of SrcID to that of RefID.
SetPreference[a,b,c](w, x, y, z)	Override Preferences settings with options a,b,c and values w, x, y, z.
RedID, GreenID, BlueID = **Split**[a](SrcID)	Split SrcID into specified images using option a.
Suspend( )	Suspend execution of the script.

Scripting directives

The directives which may appear in a script file that is specified in an Execute Script command are listed below. Each directive must be written exactly as indicated, including case and with the leading exclamation mark (!). The exclamation mark must appear at the start of a line, in column 1.

!IgnoreWarnings	Specify that the script should run even if warnings (missing images) are encountered during the validation phase. By default, a script will not run if it has any validation-time warnings. See Execute Script for details on script validation.
!SingleStep	Suspend after each script command is executed. To resume a suspended script, use the Resume Script command or CTRL + E.
!NormalStep	Revert from SingleStep mode to normal operation.

The !SingleStep and !NormalStep directives take effect when they are encountered. Any commands before the first !SingleStep execute normally, as do any commands after a !NormalStep until the next !SingleStep (or the end of the script). The !IgnoreWarnings directive can appear anywhere in the script.

Sample scripts

Click on the sample scripts below to see examples of script files. To use one of these samples as a template for your own script:

1. Create a new text file that contains a copy of the sample script. See the *Scripts* subdirectory of the RegiStar installation directory for original text files that contain these scripts. You can copy one of these scripts from this directory, or copy the script from the RegiStar Help system and paste it to a new text file.
2. Replace the generic directory and file names specified within the quotation marks "..." in the Label commands at the top of the script with your actual names.
3. Make other necessary changes to the script.
4. Save the edited script file as a .txt file.
5. Run the Execute Script command, specifying this script file as the input.

Sample scripts:

- Preprocess script
- Register script
- Mosaic script
- Overlay script
- Tricolor script
- Stacking script
- RGB realignment script for nonraw images
- RGB realignment script for raw images

Note: The evaluation version of RegiStar has saving disabled. You must remove or comment out any Save, SaveAs, or SaveAll commands in these scripts in order to run them under the evaluation version of RegiStar. Also, your script must not include any bulk command that requests saving of output images.

Command-line scripting

You can run a script by typing a command line in a Windows Command Prompt window, instead of using the Execute Script command in a RegiStar session.

If the RegiStar executable is in your path, or you are in the directory containing the RegiStar executable, enter:

```
registar -script <script file path>     (1)
```

where `<script file path>` is the absolute path of the script file you wish to run (for example, `C:\scripts\script1.txt`) or the path relative to the current directory (for example, `..\scripts\script1.txt`). If this path contains any spaces, you must enclose the path in quotation marks (for example, `"C:\script files\script1.txt"`).

If you are not in the directory containing the RegiStar executable and it is not in your path, replace `registar` in (1) with the absolute or relative path name of your RegiStar executable, for example, `"C:\Program Files\Auriga Imaging\RegiStar\registar"` or `"..\..\Program Files\Auriga Imaging\RegiStar\registar"`. As above, you must use quotation marks if the path includes spaces.

When a script is run using a command line, a RegiStar session is started and this session is ended once the script is complete. If a RegiStar session is already running, then that session is brought to the foreground and the script is not run.

You can run several scripts one after the other by writing a batch file (.bat) containing command lines for the various scripts that you want to run.

RegiStar returns a *return code* after running any script. These codes are as follows:

- 0: normal termination
- 1: couldn't open the script file or corresponding log file
- 2: script was interrupted by user
- 3: script failed validation
- 4: script suffered a runtime error

You can use these codes to specify subsequent actions in a batch file. For example:

```
registar -script script1.txt
if not errorlevel 1 registar -script script2.txt
```

would only start the execution of `script2.txt` if the execution of `script1.txt` did not return a code of 1. If you replace the second line with `if errorlevel 0...`, `script2.txt` would execute only if `script1.txt` ran normally.